

Kaori IMD: A Source-Aware Workspace and a Budgeted Model for Agent-Task Reimbursement

An engineering study of available utilities and the proposed Kaori Fuel architecture

Kaori IMD Project 5 October 2026

Project-authored research. Not peer reviewed. This paper describes available Kaori tools and the proposed Kaori Fuel design.

Keywords IMD · Ethereum · receipts · permissions · staking · agent observability · reimbursement

01

Practical solutions for IMD begin with readable evidence, clear wallet permissions and a cost model that respects its funding.

Abstract

Kaori IMD is an independent Ethereum application that organizes practical IMD tools in a manga, pixel and comic interface. Its available workspace includes transaction receipt inspection, personal receipt notes, a spending-permission check, wallet-connected access to the official IMD staking vault, and observation of public agent, job and oracle records. The project addresses an interface problem: users need to connect payment records, wallet permissions and agent activity without confusing a displayed estimate or an external report with a confirmed transaction.

This article also specifies Kaori Fuel, a proposed separate reimbursement service intended to reduce the effective cost of eligible paid IMD tasks. The initial policy returns 50% of a verified service payment, protects 20% of each confirmed funding receipt and reserves liabilities before accepting coverage. Funding would come from IMD rewards claimed by the project for its own Stockereum holdings and subsequently transferred to the Fuel vault. Other holders retain their own rewards. The contribution is an inspectable system description and a finite-budget accounting model. Fuel remains a design: its contract, payment verifier, transaction automation and activation have not been implemented or deployed. No conclusion about adoption, profitability, security certification or unlimited operation follows from this study.

Project status at publication

Component	Status and boundary
Receipt Desk and personal archive	Implemented; public Ethereum readings and browser-local records
Approval Check	Implemented; reads one IMD owner/spender allowance without a signature
IMD Staking Desk	Implemented; visitor-authorized transactions in the official POOL4 vault
Network Desk and watchlist	Implemented; public IMD records and browser-local references
Kaori Fuel	Funding policy defined; contract and automation not deployed
Proprietary Kaori token address	No active address configured in the inspected application
Independent security audit	Not established by this article

1. Background and Research Question

IMD users encounter several kinds of evidence: an Ethereum payment receipt, permission granted to a contract, a vault share balance, and an agent service's published work status. These records answer different questions. A receipt can confirm a transfer; an allowance describes spending permission; a public job record reports a service state. None is interchangeable with the others.

Kaori brings these records into one workspace while retaining their source and time. Its visual identity supplies a familiar navigation language; the practical utility comes from the underlying reads and explicitly chosen wallet actions. The project is independent of Identity.md and POOL4. Using official IMD data and contracts does not imply official affiliation.

The research question has two parts: how can a personal IMD workspace make existing activity easier to inspect, and how could a separately funded service lower eligible agent-task costs without accepting unfunded obligations? The first part concerns the implemented application. The second concerns the proposed Fuel design. The article keeps those implementation stages separate.

2. Materials and Methods

The materials were the Kaori frontend and server modules, its source-boundary document, its Fuel funding policy and primary Ethereum, IMD, POOL4 and Stockereum documentation. The inspection examined how displayed information maps to source records, how wallet actions are limited, where information is stored and how proposed reimbursement liabilities would be admitted. Source files and protocol references are listed below. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#) [\[6\]](#) [\[7\]](#) [\[8\]](#) [\[9\]](#) [\[10\]](#)

This is a descriptive engineering study rather than a controlled user experiment. The methods were source inspection, documentation comparison and arithmetic analysis of the funding policy. No new token deployment, transfer, approval, staking deposit or reimbursement was submitted for this article. No wallet balance, trading volume, participant count or security outcome is manufactured as a research result.

Implementation statements refer to the inspected modules. External documentation describes the relevant interfaces and economic routing, but does not establish the correctness of Kaori's complete implementation. The article reports no longitudinal availability measurement, independent audit or experimentally measured adoption benefit.

3. Available Utilities and the Problems They Address

3.1 Receipt Desk: understanding an IMD transfer

An explorer entry can be difficult to connect to a user's own purpose. Receipt Desk accepts a transaction hash and retrieves its Ethereum transaction, receipt, block and official IMD transfer events. It exposes the sender, recipient, status, execution gas fee and observed block information, with official IMD amounts preserved exactly. A user can add a personal note and keep the reading in an archive. [\[1\]](#) [\[3\]](#) [\[11\]](#)

The read service checks agreement between transaction and receipt fields and verifies their canonical block. It distinguishes a pending record, a reverted transaction and a successful Ethereum transaction with no official IMD transfer. A valid Ethereum receipt is not automatically classified as a token purchase, sale, profit or completed agent task. Where the provider supplies usable finality information, that observation is distinguished from the confirmation count. [\[1\]](#)

The archive keeps up to 30 records in the visitor's browser. Notes can be exported and restored; reopening a saved hash requests a fresh Ethereum reading. A saved reading retains its original observation time. Browser storage is personal storage, not a shared permanent database, and a backup is needed to move notes between devices. [\[1\]](#) [\[3\]](#)

3.2 Approval Check: making spending permission visible

Token spending permission can remain after its original purpose has ended. Approval Check reads the official IMD allowance for one wallet and one specified spender. It returns the exact remaining token permission and the block used for the observation. Only the maximum unsigned 256-bit value is identified as maximum approval; a large finite allowance is still finite. [\[1\]](#) [\[4\]](#) [\[6\]](#)

This tool addresses awareness of a particular permission. It does not discover every spender, prove the safety of the spender, read an allowance as a token balance or revoke permission. It requires neither wallet connection nor a signature. An unavailable response remains unavailable rather than being converted into zero permission. [\[1\]](#) [\[4\]](#)

3.3 Staking Desk: access to the official IMD vault

The staking workspace connects the visitor's chosen browser wallet to the official Ethereum POOL4 sIMD vault. IMD is the deposited asset and sIMD represents vault shares; it is a distinct official protocol integration, not a proprietary Kaori staking pool. ERC-4626 defines the underlying asset/share interface. [\[1\]](#) [\[5\]](#) [\[7\]](#) [\[8\]](#) [\[12\]](#)

Kaori reads wallet balances, allowance, vault conversion values and current transaction limits. The inspected implementation checks official contract identity and uses 18 asset decimals and 24 share decimals. A deposit requires approval for the entered amount when necessary and a separate wallet-confirmed deposit. Redemption sends the underlying asset to the connected account. The server does not sign these actions or take custody of the wallet. [1] [5]

Before a submission, the interface checks the account, network and current contract state, then simulates the action and estimates gas. Confirmation requires the matching transaction intent, a canonical successful receipt, the expected official event and two confirmations. A returned transaction hash alone is not success. Actual event amounts replace previews after confirmation. [1] [5]

Previewed share or redemption quantities may change before execution. Kaori claims no fixed APR or guaranteed gain. The official vault has a one-block redemption hold; POOL4 documentation also describes its contract risks and ownership powers. Kaori reads current ownership and pause state rather than assuming those powers are active or renounced. [1] [7]

3.4 Network Desk: following the work behind IMD

Network Desk reads public IMD agent, job and oracle records. Visitors can inspect a bounded list or open a specific record, retain references with personal notes and export a watchlist. It gives the source and retrieval time so a saved item can be reopened for a newer observation. [1] [9]

The official API supplies public job and oracle routes. The IMD documentation distinguishes an accepted job verdict from a signed oracle result, and publishes the signed-answer interface separately. Kaori presents the reported fields; its current reader does not independently certify a task's correctness or verify an oracle signature. A historical seat record is not proof that an agent is presently online. [1] [9]

The reader does not open paid work, trade tokens, distribute rewards or move treasury funds. When an upstream service is unavailable, the interface reports that state. This utility improves visibility into activity while preserving the distinction between observation and execution. [1]

4. Kaori Fuel: Proposed Service-Cost Reimbursement

Paid agent use creates a recurring service cost for builders. Fuel's proposed contribution is a budgeted partial reimbursement for an eligible official IMD service payment. It is not an earnings promise, payment for merely asking a question or reimbursement for every Ethereum transfer. Coverage would require a valid reservation and a verified paid order. [2]

The initial rate is 50%. For an eligible payment expressed in the token's smallest unit, p , the proposed reimbursement is:

$$\text{reimbursement} = \text{floor}(p \times 5,000 / 10,000)$$

The confirmed order amount supplies p ; the service price is not hard-coded. Rounding is down to the smallest token unit. Payment goes to the wallet that made the verified service payment, and an order can receive coverage only once. Network gas is outside this service-cost reimbursement. [2]

Fuel would use a contract separate from the Kaori token and from the official staking vault. The contract would custody its own funding and enforce accepted liabilities. A verifier would establish the link between a reservation, a canonical payment, its payer and the eligible order. A transaction executor would submit authorized payouts. Merely observing a completed job or accepting a user's claim would not establish reimbursement authorization. [2]

IMD's paid-request documentation supports wallet-authorized payments and public request or job inspection. Its signed oracle interface could inform a future proof design, but does not itself establish that Fuel's specific eligibility requirements are verified. The contract's proof format, authorized signer, supported order types and settlement rules still require implementation and review. [9]

5. Funding, Reserve Accounting and Sustainability

5.1 Funding ownership

The selected funding route is manual treasury contribution: the project claims the IMD rewards attributable to its own Kaori holdings on Stockereum and transfers those received assets to Fuel. Other holders' rewards belong to those holders and remain outside this budget. Funding becomes usable only after confirmed receipt by a deployed vault. [2]

Stockereum documentation includes IMD among supported paired assets and describes fees denominated in the paired asset. It permits the creator share to be directed to holders proportionally, with that routing fixed at launch. This description does not prove a particular Kaori launch, balance, claim or deposit. Those events would need separate chain evidence. [10]

5.2 Protecting deposits and avoiding double commitment

The policy protects 20% of each confirmed funding receipt. For a new deposit d :

$$\text{protected addition} = \text{floor}(d \times 2,000 / 10,000)$$

The reserve is derived from actual funding receipts. It is not repeatedly recalculated as 20% of a shrinking balance, and repeated processing or internal transfers must not manufacture new funding. The user-paid half of a service charge is not income received by Kaori. [2]

Let B be confirmed IMD assets held by Fuel, S its protected reserve, L outstanding accepted reimbursements and C committed IMD verification costs. Available admission budget is:

$$\text{free budget} = B - S - L - C$$

A proposed reservation is admissible only when its reimbursement and verification cost fit the free budget, with a separately sufficient ETH transaction budget. These funds must be committed before coverage is accepted, so concurrent requests cannot all rely on the same balance. New coverage stops when admission funds are insufficient; eligible existing commitments retain their promised amount. Later rate changes apply to new reservations. [2]

5.3 Conditions for continuing operation

The 50% reimbursement rate and 20% reserve are initial policy choices, not a mathematical guarantee of lifetime. Over an interval, continued operation without consuming earlier free funds requires new usable IMD funding to cover reimbursements and IMD verification expenses. With I as newly confirmed funding, R as payouts and V as verification costs, the long-term admission condition is approximately:

$$0.80 \times I \geq R + V$$

Small-unit rounding and existing commitments must be handled exactly in implementation. ETH inflows must separately cover execution gas and other ETH expenses. Tokens and ETH cannot be combined into one budget without an explicit conversion and price model. Lower trading activity, higher usage or greater verification costs can therefore close new coverage even with the selected percentages. [2]

Paying for a separate oracle request for every small reimbursement could exceed the subsidy itself. A server verifier or batching approach may reduce cost, but introduces its own trust and review requirements. Automatically paying reimbursements still needs an available executor, a funded gas balance and transactions confirmed on Ethereum. Automatic execution does not create new revenue. [2]

6. Verification Requirements and Threat Model

Fuel must bind each claim to one authorized reservation and eligible order, the original payer, the official payment asset, the actual settled amount and its intended contract and chain. It must reject duplicate claims and evidence that is incomplete, stale or inconsistent. Provider-side refunds and unsuccessful settlements must not be counted as an unreduced eligible expense. These are design requirements for implementation; the present article does not establish that a deployed contract enforces them. [2]

A cryptographic signature establishes who signed a statement. It does not independently establish that every statement is true. A server-verifier model would therefore place trust in its authorized signer and the evidence procedure. An oracle model would instead need to define the trusted attester, exact question, proof format, expiry and agreement requirements. Neither model should grant an agent unrestricted authority to transfer vault assets. [2] [9]

Duplicate-order protection alone does not prevent one person using many wallets. Future admission rules should account for automated demand, repeated equivalent tasks and disproportionate budget consumption. The launch-ready design must also specify reservation deadlines, settlement handling, signer replacement, emergency controls and treatment of existing obligations. None of these administrative choices is presented here as deployed or finalized.

7. Limitations and Scope of the Findings

Receipt and allowance reads depend on external Ethereum RPC availability. Network records depend on IMD's public service, and browser-local archives depend on available storage. Canonical-block checks and validation address particular inconsistencies; they do not guarantee that every external provider is correct or always reachable. A refreshed record can differ from an earlier saved observation. [1]

Official staking carries the risks of its own contracts and market. Fuel would add separate contract, verifier, operational and funding dependencies. This article does not establish an audit of either integration, guarantee token value or confirm a source of perpetual revenue. Its equations express conservative admission accounting rather than a claim of profitability.

The observed contribution is a coherent workspace with traceable information and limited wallet actions. Fuel's contribution is currently a written reimbursement architecture. A future report could evaluate confirmed funding, settled payments, verification costs, availability and budget exhaustion using published evidence. No such performance results are claimed here.

8. Conclusion

Kaori IMD provides practical tools for understanding transfers, preserving personal notes, checking a specified spending permission, accessing official IMD staking and following public agent work. It addresses the usability gap between raw records and the evidence a user needs before acting.

The proposed Kaori Fuel service addresses a different constraint: the cost of repeated paid agent tasks. Its initial design reimburses half of an eligible service payment while protecting part of each funding deposit and committing liabilities before coverage is accepted. This would be a finite subsidy supplied by real IMD contributions, with separately funded execution costs. Contract deployment, evidence verification, automation and activation remain necessary before that proposed utility can be used.

References and Source Record

- [1] **Kaori IMD: Sources and data boundaries.** Project engineering record, `docs/SOURCES.md`, inspected 5 October 2026.
[Open source ↗ - https://github.com/x80zAI/kaori-imd/blob/blcec2d5dfafd75d0c7cc84742729b1444013911/docs/SOURCES.md](https://github.com/x80zAI/kaori-imd/blob/blcec2d5dfafd75d0c7cc84742729b1444013911/docs/SOURCES.md)
- [2] **Kaori IMD: Fuel funding policy.** Project policy dated 5 October 2026. The document defines proposed rules, not a live deposit address.
[Open source ↗ - https://kaorimd.site/research/kaori-fuel-policy.txt](https://kaorimd.site/research/kaori-fuel-policy.txt)
- [3] **Kaori IMD: Receipt and archive implementation.** `src/ReceiptInspector.tsx`, `src/domain.mjs`, `src/useArchive.ts`, `src/Archive.tsx` and server receipt modules, inspected 5 October 2026.
[Open source ↗ - https://github.com/x80zAI/kaori-imd/tree/blcec2d5dfafd75d0c7cc84742729b1444013911/src](https://github.com/x80zAI/kaori-imd/tree/blcec2d5dfafd75d0c7cc84742729b1444013911/src)
- [4] **Kaori IMD: Approval Check implementation.** `src/ApprovalCheck.tsx`, `src/approval.mjs` and server allowance module, inspected 5 October 2026.
[Open source ↗ - https://github.com/x80zAI/kaori-imd/tree/blcec2d5dfafd75d0c7cc84742729b1444013911/src](https://github.com/x80zAI/kaori-imd/tree/blcec2d5dfafd75d0c7cc84742729b1444013911/src)
- [5] **Kaori IMD: Staking implementation.** `src/StakingDesk.tsx`, `src/staking.mjs`, `src/staking-wallet.mjs` and server staking module, inspected 5 October 2026.
[Open source ↗ - https://github.com/x80zAI/kaori-imd/tree/blcec2d5dfafd75d0c7cc84742729b1444013911/src](https://github.com/x80zAI/kaori-imd/tree/blcec2d5dfafd75d0c7cc84742729b1444013911/src)
- [6] **ERC-20: Token Standard.** Ethereum Improvement Proposal 20.
[Open source ↗ - https://eips.ethereum.org/EIPS/eip-20](https://eips.ethereum.org/EIPS/eip-20)
- [7] **POOL4: Official staking, parameters and risk documentation.**
[Open source ↗ - https://pool4.imd.fun/docs#staking](https://pool4.imd.fun/docs#staking)
- [8] **ERC-4626: Tokenized Vaults.** Ethereum Improvement Proposal 4626.
[Open source ↗ - https://eips.ethereum.org/EIPS/eip-4626](https://eips.ethereum.org/EIPS/eip-4626)
- [9] **Identity.md: Official API documentation.** Public jobs, oracle signatures and paid requests.
[Open source ↗ - https://imd.fun/docs/](https://imd.fun/docs/)
- [10] **Stockereum: How it works.** Paired-asset fees and holder-reward routing.
[Open source ↗ - https://stockereum.com/docs](https://stockereum.com/docs)
- [11] **Ethereum: JSON-RPC API documentation.**
[Open source ↗ - https://ethereum.org/developers/docs/apis/json-rpc/](https://ethereum.org/developers/docs/apis/json-rpc/)
- [12] **StakedIMD: Verified official Ethereum contract source.**
[Open source ↗ - https://etherscan.io/address/0x9efa934d9fad4ae28c998a40195646b965a97247#code](https://etherscan.io/address/0x9efa934d9fad4ae28c998a40195646b965a97247#code)

External protocol documentation was consulted on 5 October 2026. The project-source revision is recorded for reproducibility; this article's date does not imply ongoing availability or a completed independent audit.